Data_Research_Project_Report

April 21, 2026

[68]:
```python
# Import libraries
import pandas as pd
import matplotlib.pyplot as plt

# Import excel as CSV
annual_earnings_by_education_attainment = pd.
 ↪read_csv('MedianEducationalEarnings.csv', encoding='latin-1')
annual_earnings_by_education_attainment
```

[68]:
```
   Sex, race/ethnicity, and educational attainment     2005  2005.1  \
0     Median annual earnings, all education levels  $48,990    -582
1                Less than high school completion    32,130    -442
2                           High school completion\1\    41,530    -644
3                         Some college, no degree    47,640    -321
4                               Associate's degree    52,530  -1,113
5                       Bachelor's or higher degree    66,940  -1,320
6                                Bachelor's degree    62,340    -435
7                         Master's or higher degree    77,090    -736
8                Percent, all education levels\2\      70.8   -0.36
9                Less than high school completion      64.1   -1.29
10                          High school completion\1\      71.3   -0.71
11                        Some college, no degree      67.3   -0.82
12                              Associate's degree      71.8   -1.09
13                      Bachelor's or higher degree      74.4   -0.67
14                               Bachelor's degree      75.3   -0.76
15                        Master's or higher degree        72   -1.18

       2010  2010.1     2012  2012.1     2013  2013.1     2014  …  \
0  $50,290    -433  $49,670    -665  $50,790    -204  $49,740  …
1    29,330    -428   29,480    -859   30,580  -1,095   30,720  …
2    41,460    -299   39,350    -298   38,830    -327   38,560  …
3    44,030  -1,103   42,130  -1,203   44,020  -1,115   40,080  …
4    50,190  -1,201   46,970    -735   47,620  -2,470   44,740  …
5    66,850    -963   64,990    -352   64,930    -394   64,760  …
6    62,290    -574   60,340    -877   62,000  -1,445   62,690  …
7    75,320  -1,115   77,200  -1,346   76,660    -743   74,410  …
8      64.8   -0.42     67.2   -0.43     67.8    -0.5     70.2  …
```

```
9       48.1   -1.44      52.6   -1.32      55.9   -1.65      59.1  …
10        60   -0.79      63.4   -0.77      64.5   -1.09      68.4  …
11      60.9   -0.89      62.1      -1      61.9   -1.12      64.6  …
12      65.9   -1.25      67.9   -1.14      70.2   -1.47      69.2  …
13      74.3    -0.6      75.5    -0.6      74.8   -0.69      76.3  …
14      74.4    -0.7      75.7   -0.74      75.1   -0.83      76.4  …
15      73.8   -1.22      75.2   -1.15        74   -1.27      76.2  …

   2018.1      2019  2019.1      2020  2020.1      2021  2021.1      2022  \
0    -763  $53,950     -427  $57,510     -226  $55,440     -245  $54,210
1  -1,721    35,020    -640    35,270    -685    35,330  -2,331    35,470
2    -440    41,280    -416    42,140    -641    43,970    -336    41,790
3    -460    45,540  -1,017    46,110    -427    44,980    -381    45,230
4    -502    47,350    -511    50,620  -1,560    49,370    -931    49,470
5  -1,240    69,500    -512    70,310    -457    71,650    -638    71,060
6    -550    64,880    -622    68,470    -565    67,320    -776    66,610
7  -1,169    81,660  -1,339    79,720  -1,391    81,460  -1,834    80,180
8    -0.4      74.5   -0.39      65.7   -0.45      72.3    -0.4      75.8
9   -1.78        65   -2.09      49.1   -1.98        53   -2.17      63.8
10  -0.86      71.8   -0.86      59.7   -0.79      67.9   -0.85      73.2
11  -0.92      69.4   -0.99      58.4   -1.06      70.1   -1.17      71.3
12  -1.14      73.5   -1.05      62.1    -1.3      69.5   -1.38      76.2
13  -0.56      79.2   -0.58      74.4   -0.64      78.4   -0.57      79.9
14  -0.66      78.9   -0.73      73.3   -0.78      78.5    -0.7      79.8
15  -0.94      79.9      -1      77.1   -1.05      78.1   -0.97      80.2

   2022.1  2022.2
0     NaN  -1,073
1     NaN    -975
2     NaN    -254
3     NaN  -1,135
4     NaN  -1,089
5     NaN    -428
6     NaN    -788
7     NaN  -1,318
8     NaN   -0.39
9     NaN   -2.44
10    NaN   -0.87
11    NaN    -1.1
12    NaN   -1.17
13    NaN   -0.58
14    NaN   -0.71
15    NaN   -0.94

[16 rows x 28 columns]
```

1. **Research Question → "Asking an interesting question"** How does the level of educational

attainment affect median income across U.S. states? The question may seem insignificant at first; however, it is becoming increasingly important to examine as we become an increasingly technology-based society. Technology has opened up several avenues for generating income- many of which don't require an education at all. However, in many career spaces it has further pronounced the disparity between skilled and unskilled labor, which is coined as "Skill-Biased Technological Change". Essentially, as technology develops, it rises in complexity, which raises not only the ceiling for skilled labor, but the floor, which ends up cutting out lower skilled labor, i.e. replacing a worker with AI or some automated system that can be operated by a single person. Naturally, as one becomes more educated, their skills should generally increase in tandem. To conclude, the question may seem obvious, but the full picture is likely of a higher magnitude than one would expect.

2. Data acquisition → "Obtaining the data" Data Source Citation: National Center for Education Statistics. (2024, May). COE - Annual Earnings by Educational Attainment. Nces.ed.gov; National Center for Education Statistics. https://nces.ed.gov/programs/coe/indicator/cba/annual-earnings

This data came as an excel but was converted to csv for ease of importing through pandas.

This data was pulled from 25-35 year olds who worked full-time, year round. The dataset examines changes in earnings across the following education attainment classifications: "Less than high school completion", "high school completion", "some college, no degree", "Associate's degree", "Bachelor's or higher degree", "Bachelor's degree", "Master's or higher degree". There are classifications for specific races, but for the purposes of this project, only the totals will be used to get the bigger picture. Each column represents a year, and within each column are the earnings and standard error.

3. Data exploration → "Exploring the data" Using the info() method, we can already see the usefulness of the data, as it provides a large timeline's worth of data, which will provide the big picture snapshot needed to understand the magnitude change over the years. For the purposes of the project, 2005-13 and 2018-2022 will be used, as it still captures the major periods for the project.

[69]:
```
annual_earnings_by_education_attainment.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 16 entries, 0 to 15
Data columns (total 28 columns):
 #   Column                                       Non-Null Count  Dtype
---  ------                                       --------------  -----
 0   Sex, race/ethnicity, and educational attainment  16 non-null     object
 1   2005                                         16 non-null     object
 2   2005.1                                       16 non-null     object
 3   2010                                         16 non-null     object
 4   2010.1                                       16 non-null     object
 5   2012                                         16 non-null     object
 6   2012.1                                       16 non-null     object
 7   2013                                         16 non-null     object
 8   2013.1                                       16 non-null     object
 9   2014                                         16 non-null     object
 10  2014.1                                       16 non-null     object
```

```
 11  2015                                                 16 non-null     object
 12  2015.1                                               16 non-null     object
 13  2016                                                 16 non-null     object
 14  2016.1                                               16 non-null     object
 15  2017                                                 16 non-null     object
 16  2017.1                                               16 non-null     object
 17  2018                                                 16 non-null     object
 18  2018.1                                               16 non-null     object
 19  2019                                                 16 non-null     object
 20  2019.1                                               16 non-null     object
 21  2020                                                 16 non-null     object
 22  2020.1                                               16 non-null     object
 23  2021                                                 16 non-null     object
 24  2021.1                                               16 non-null     object
 25  2022                                                 16 non-null     object
 26  2022.1                                               0 non-null      float64
 27  2022.2                                               16 non-null     object
dtypes: float64(1), object(27)
memory usage: 3.6+ KB
```

Next, use describe() gives a list of basic summary statistics for the data. Looking closely at the data, one will notice that the biggest change going year to year was the max going from 2018 to 2019. This is one coincidence, as this period marked a significant leap in AI, taking it from proof of concept to actual practical application. Naturally, the max represents the top level of education (skill), so this is consistent.

[70]:
```
# Fills NaNs with 0
annual_earnings_by_education_attainment = annual_earnings_by_education_attainment.fillna(0)

# List of columns
std_error_cols = ['2005.1', '2010.1', '2012.1', '2013.1', '2018.1', '2019.1', '2020.1', '2021.1', '2022.1']
yearly_earnings_cols = ['2005', '2010', '2012', '2013', '2018', '2019', '2020', '2021', '2022']
education_levels_col = 'Sex, race/ethnicity, and educational attainment'

# Converts every column entry into an integer for proper plotting
for col in annual_earnings_by_education_attainment:
  # If the column is the single string column, skip it
  if col == education_levels_col:
    continue
  # Replace any bad characters explicitly as string to avoid error. Eliminates possibility of pandas
  # thinking the data entries are objects instead of numerical data
  annual_earnings_by_education_attainment[col] = (
        annual_earnings_by_education_attainment[col]
        .astype(str)
```

```
        .str.replace('$', '', regex=False)
        .str.replace(',', '', regex=False)
        .str.replace('-', '', regex=False)
        .str.strip()
    )
  # Make the entries into int type
  annual_earnings_by_education_attainment[col] = pd.
↪to_numeric(annual_earnings_by_education_attainment[col])
```

[71]:
```
annual_earnings_by_education_attainment.describe()
```

[71]:
```
               2005       2005.1          2010       2010.1          2012  \
count      16.00000     16.00000     16.000000    16.000000     16.000000
mean    26859.81250    349.99250  26267.637500   382.706875  25666.850000
std     29385.96438    432.36139  28923.145045   467.313509  28460.050518
min        64.10000      0.36000     48.100000     0.420000     52.600000
25%        71.67500      0.80500     65.625000     0.865000     67.725000
50%     16102.65000    161.14500  14702.200000   150.220000  14777.850000
75%     49875.00000    597.50000  50215.000000   671.250000  47645.000000
max     77090.00000   1320.00000  75320.000000  1201.000000  77200.000000

             2012.1        2013       2013.1          2014       2014.1  …  \
count     16.000000     16.0000    16.000000     16.000000    16.000000  …
mean     396.384375  25998.3875   487.601250  25391.275000   317.935625  …
std      479.386475  28693.7782   715.678498  28116.478428   414.150042  …
min        0.430000     55.9000     0.500000     59.100000     0.390000  …
25%        0.942500     69.6000     1.112500     69.950000     0.815000  …
50%      149.660000  15327.5500   102.825000  15398.200000   106.715000  …
75%      766.000000  48412.5000   831.000000  45990.000000   571.250000  …
max     1346.000000  76660.0000  2470.000000  74410.000000  1364.000000  …

            2018.1          2019       2019.1         2020       2020.1  \
count     16.00000     16.000000    16.000000     16.00000    16.000000
mean     428.26625  27485.762500   343.230625  28166.86250   372.503125
std      546.09469  30306.814622   417.726946  30885.44924   502.655510
min        0.40000     65.000000     0.390000     49.10000     0.450000
25%        0.90500     74.250000     0.957500     64.80000     0.985000
50%      220.89000  17549.950000   209.045000  17673.55000   113.990000
75%      603.25000  49000.000000   539.500000  52342.50000   584.000000
max     1721.00000  81660.000000  1339.000000  79720.00000  1560.000000

                2021       2021.1          2022  2022.1       2022.2
count      16.000000    16.000000     16.000000    16.0    16.000000
mean    28130.487500   467.513125  27788.762500     0.0   441.762500
std     30936.546637   706.828327  30546.283034     0.0   520.218766
min        53.000000     0.400000     63.800000     0.0     0.390000
25%        71.750000     0.940000     76.100000     0.0     0.922500
```

```
50%     17704.250000   123.585000  17775.100000     0.0   128.220000
75%     50887.500000   672.500000  50655.000000     0.0   999.500000
max     81460.000000  2331.000000  80180.000000     0.0  1318.000000

[8 rows x 27 columns]
```

Calling isna() returns false for all columns. It is important to note that if NaNs weren't converted to zeros earlier that the 2022.1 column would be flagged. This column was empty because data on the std error for 2022 could not be collected. However, this weakness is at the very end of the data set, and thankfully doesn't fall in the middle of a critical period.

[72]:
```python
annual_earnings_by_education_attainment.isna().head(5)
```

[72]:
```
   Sex, race/ethnicity, and educational attainment   2005  2005.1   2010  \
0                                            False  False   False  False
1                                            False  False   False  False
2                                            False  False   False  False
3                                            False  False   False  False
4                                            False  False   False  False

   2010.1   2012  2012.1   2013  2013.1   2014  …  2018.1   2019  2019.1  \
0   False  False   False  False   False  False  …   False  False   False
1   False  False   False  False   False  False  …   False  False   False
2   False  False   False  False   False  False  …   False  False   False
3   False  False   False  False   False  False  …   False  False   False
4   False  False   False  False   False  False  …   False  False   False

    2020  2020.1   2021  2021.1   2022  2022.1  2022.2
0  False   False  False   False  False   False   False
1  False   False  False   False  False   False   False
2  False   False  False   False  False   False   False
3  False   False  False   False  False   False   False
4  False   False  False   False  False   False   False

[5 rows x 28 columns]
```

Calling the nunique() method shows that there are NO duplicate values except for the NaNs that were converted to zeros in the 2022.1 column. Additionally, if the column with the education levels was included, with the percent versions still in, it would flag duplicates as row names are repeated for education levels. This is a good thing, as it shows the quality of the data we're working with, coming with another form of the existing data from the start.

[73]:
```python
annual_earnings_by_education_attainment.nunique().tail(5)
```

[73]:
```
2021      16
2021.1    16
2022      16
2022.1     1
```

```
2022.2    16
dtype: int64
```

The shape attribute, similar to describe, shows the wide timeline of the data, emphasizing the big picture goal.

[74]:
```
annual_earnings_by_education_attainment.shape
```

[74]:
```
(16, 28)
```

The dtypes attribute shows the values in the columns are all floats, which is going to be a problem if you want to index values, so it's best to convert them to int.

[75]:
```
annual_earnings_by_education_attainment.dtypes.head(5)
```

[75]:
```
Sex, race/ethnicity, and educational attainment     object
2005                                               float64
2005.1                                             float64
2010                                               float64
2010.1                                             float64
dtype: object
```

Continuing from the unique() method earlier, the Sex, race and educational attainment column demonstrates what was described previously. The percent version of the data causes a repeat of rows(education levels), which is why each education level has a 2 next to it.

[76]:
```
# Stores education levels column
education_levels_col = 'Sex, race/ethnicity, and educational attainment'
annual_earnings_by_education_attainment[education_levels_col].nunique()
```

[76]:
```
9
```

The columns attribute once again shows the wide timeline of the data we're working with.

[77]:
```
annual_earnings_by_education_attainment.columns
```

[77]:
```
Index(['Sex, race/ethnicity, and educational attainment', '2005', '2005.1',
       '2010', '2010.1', '2012', '2012.1', '2013', '2013.1', '2014', '2014.1',
       '2015', '2015.1', '2016', '2016.1', '2017', '2017.1', '2018', '2018.1',
       '2019', '2019.1', '2020', '2020.1', '2021', '2021.1', '2022', '2022.1',
       '2022.2'],
      dtype='object')
```

4. Data cleaning/preparation → "Preparing the data for Models/Analysis" To prepare the data for analysis, I had to follow these steps: Cut out all the other rows outside of the totals: This was done by creating a new Excel with the necessary information only, as the row dedicated to numbering the columns, and the header with the title at the top of the excel messed up data operations. This also separated the std error from earnings as their own columns, which fixed another error that interfered with importing the data.

Remove random characters in row names: The original Excel had random characters that needed to be stripped from certain rows. To fix this, I simply used the replace function with regex set to False to target that specific character combination.

Remove NaNs: To fix this, I used fillna() to replace NaNs with 0, which is consistent formatting with the rest of the data. (Done earlier to avoid errors).

Remove minuses: When imported into pandas, the std. error values were interpreted as minuses, so I used replace() again to indiscriminately target any minus or possible bad characters and replaced it with empty space, and used strip() to delete lingering whitespace.

[78]:
```
# Use replace to with regex True to indiscriminately target any minus
annual_earnings_by_education_attainment[std_error_cols] =␣
↪annual_earnings_by_education_attainment[std_error_cols].replace('-', '',␣
↪regex=True)
annual_earnings_by_education_attainment['2005.1'].head(5)
```

[78]:
```
0     582.0
1     442.0
2     644.0
3     321.0
4    1113.0
Name: 2005.1, dtype: float64
```

Change data type of entries to float: within the same for loop I added a line that turns the column entries to float types with to_numeric(). (Done earlier for describe() to work)

Separate different versions of the data: this was accomplished by storing the regular version and percentage versions data in their own dataframes with iloc[]. The std error columns were placed into their own dataframe.

[79]:
```
# Separate accuracy scores and earnings scores for more clarity
yearly_accuracy_scores = annual_earnings_by_education_attainment[std_error_cols]
yearly_earnings = annual_earnings_by_education_attainment[yearly_earnings_cols].
↪iloc[:8]

# Percentage portion of dataframe
yearly_earnings_as_percent = annual_earnings_by_education_attainment.iloc[8:, 1:
↪]
yearly_accuracy_scores
```

[79]:

	2005.1	2010.1	2012.1	2013.1	2018.1	2019.1	2020.1	2021.1	\
0	582.00	433.00	665.00	204.00	763.00	427.00	226.00	245.00	
1	442.00	428.00	859.00	1095.00	1721.00	640.00	685.00	2331.00	
2	644.00	299.00	298.00	327.00	440.00	416.00	641.00	336.00	
3	321.00	1103.00	1203.00	1115.00	460.00	1017.00	427.00	381.00	
4	1113.00	1201.00	735.00	2470.00	502.00	511.00	1560.00	931.00	
5	1320.00	963.00	352.00	394.00	1240.00	512.00	457.00	638.00	
6	435.00	574.00	877.00	1445.00	550.00	622.00	565.00	776.00	
7	736.00	1115.00	1346.00	743.00	1169.00	1339.00	1391.00	1834.00	
8	0.36	0.42	0.43	0.50	0.40	0.39	0.45	0.40	
9	1.29	1.44	1.32	1.65	1.78	2.09	1.98	2.17	
10	0.71	0.79	0.77	1.09	0.86	0.86	0.79	0.85	
11	0.82	0.89	1.00	1.12	0.92	0.99	1.06	1.17	
12	1.09	1.25	1.14	1.47	1.14	1.05	1.30	1.38	
13	0.67	0.60	0.60	0.69	0.56	0.58	0.64	0.57	
14	0.76	0.70	0.74	0.83	0.66	0.73	0.78	0.70	
15	1.18	1.22	1.15	1.27	0.94	1.00	1.05	0.97	

	2022.1
0	0.0
1	0.0
2	0.0
3	0.0
4	0.0
5	0.0
6	0.0
7	0.0
8	0.0
9	0.0
10	0.0
11	0.0
12	0.0
13	0.0
14	0.0
15	0.0

[80]:
```
yearly_earnings_as_percent
```

[80]:

	2005	2005.1	2010	2010.1	2012	2012.1	2013	2013.1	2014	2014.1	…	\
8	70.8	0.36	64.8	0.42	67.2	0.43	67.8	0.50	70.2	0.39	…	
9	64.1	1.29	48.1	1.44	52.6	1.32	55.9	1.65	59.1	1.43	…	
10	71.3	0.71	60.0	0.79	63.4	0.77	64.5	1.09	68.4	0.80	…	
11	67.3	0.82	60.9	0.89	62.1	1.00	61.9	1.12	64.6	0.82	…	
12	71.8	1.09	65.9	1.25	67.9	1.14	70.2	1.47	69.2	1.16	…	
13	74.4	0.67	74.3	0.60	75.5	0.60	74.8	0.69	76.3	0.62	…	
14	75.3	0.76	74.4	0.70	75.7	0.74	75.1	0.83	76.4	0.68	…	
15	72.0	1.18	73.8	1.22	75.2	1.15	74.0	1.27	76.2	1.07	…	

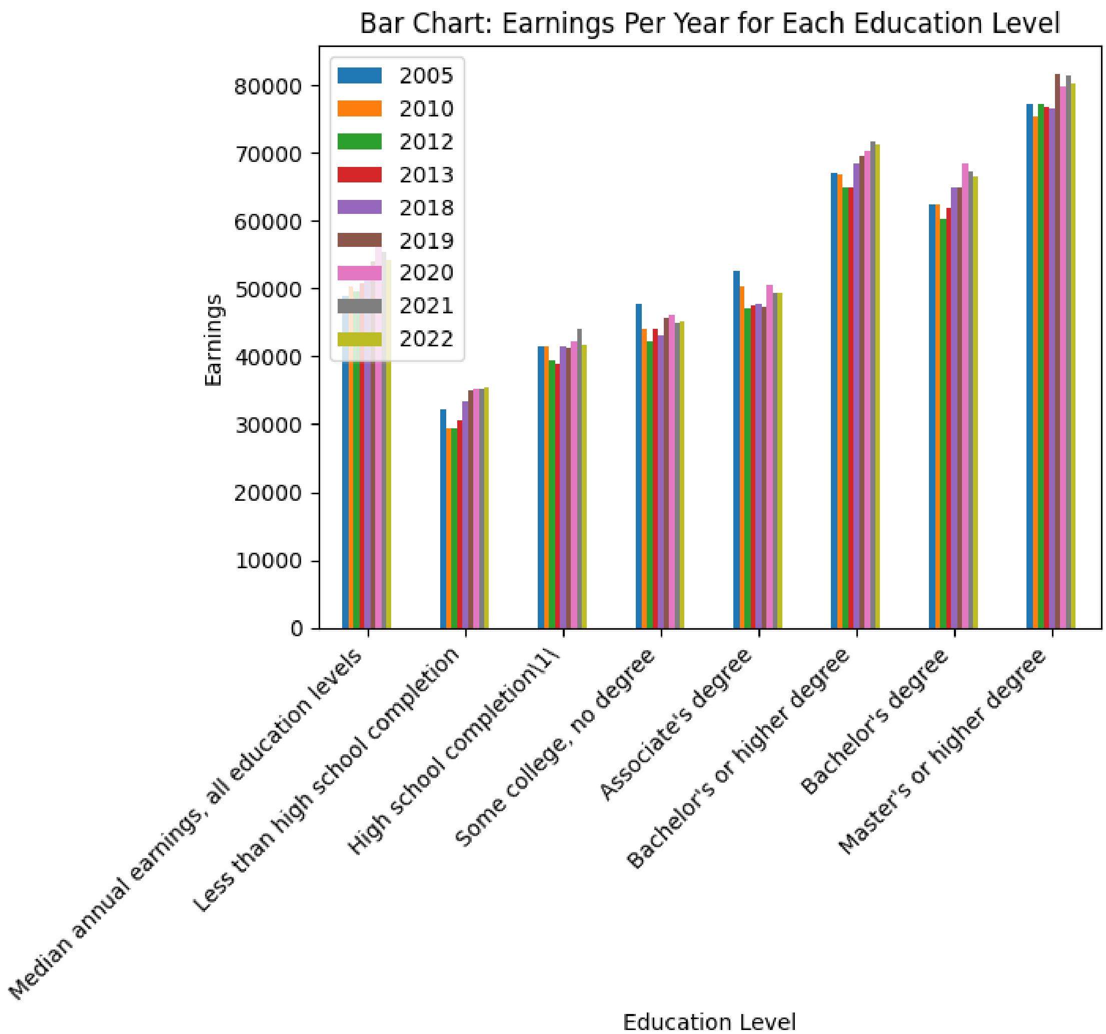
```
    2018.1  2019  2019.1  2020  2020.1  2021  2021.1  2022  2022.1  2022.2
8     0.40  74.5    0.39  65.7    0.45  72.3    0.40  75.8     0.0    0.39
9     1.78  65.0    2.09  49.1    1.98  53.0    2.17  63.8     0.0    2.44
10    0.86  71.8    0.86  59.7    0.79  67.9    0.85  73.2     0.0    0.87
11    0.92  69.4    0.99  58.4    1.06  70.1    1.17  71.3     0.0    1.10
12    1.14  73.5    1.05  62.1    1.30  69.5    1.38  76.2     0.0    1.17
13    0.56  79.2    0.58  74.4    0.64  78.4    0.57  79.9     0.0    0.58
14    0.66  78.9    0.73  73.3    0.78  78.5    0.70  79.8     0.0    0.71
15    0.94  79.9    1.00  77.1    1.05  78.1    0.97  80.2     0.0    0.94

[8 rows x 27 columns]
```

5. Data visualization → "Graphs and charts to have insights" Bar plot: The question at its root is simple: Do earnings increase with education level? The data shows that it does! There is a clear increase in earnings at each level and the years make a clear positive slope with Bachelor's or higher swapped with Bachelor's. If you were to isolate the same year across the education levels, the positive slope still holds. There is also a noticeable much larger jump going from an Associate's degree to a Bachelors. In fact, the graph's overall slope past an Associate's degree begins to resemble exponential growth. In conclusion, this bar plot demonstrates a positive correlation between earnings and education level, with a clear increase at each level in order, providing a foundation for the rest of the visualizations to build off of.

[81]:
```python
annual_earnings_by_education_attainment.iloc[:8].set_index('Sex, race/
 ↪ethnicity, and educational attainment')[yearly_earnings_cols].
 ↪plot(kind='bar', xlabel="Education Level", ylabel='Earnings')
plt.title('Bar Chart: Earnings Per Year for Each Education Level')

# Have labels rotated and set right end of ticks as center to make the plot␣
 ↪look cleaner
plt.xticks(rotation=45, ha= 'right')
plt.show()
```

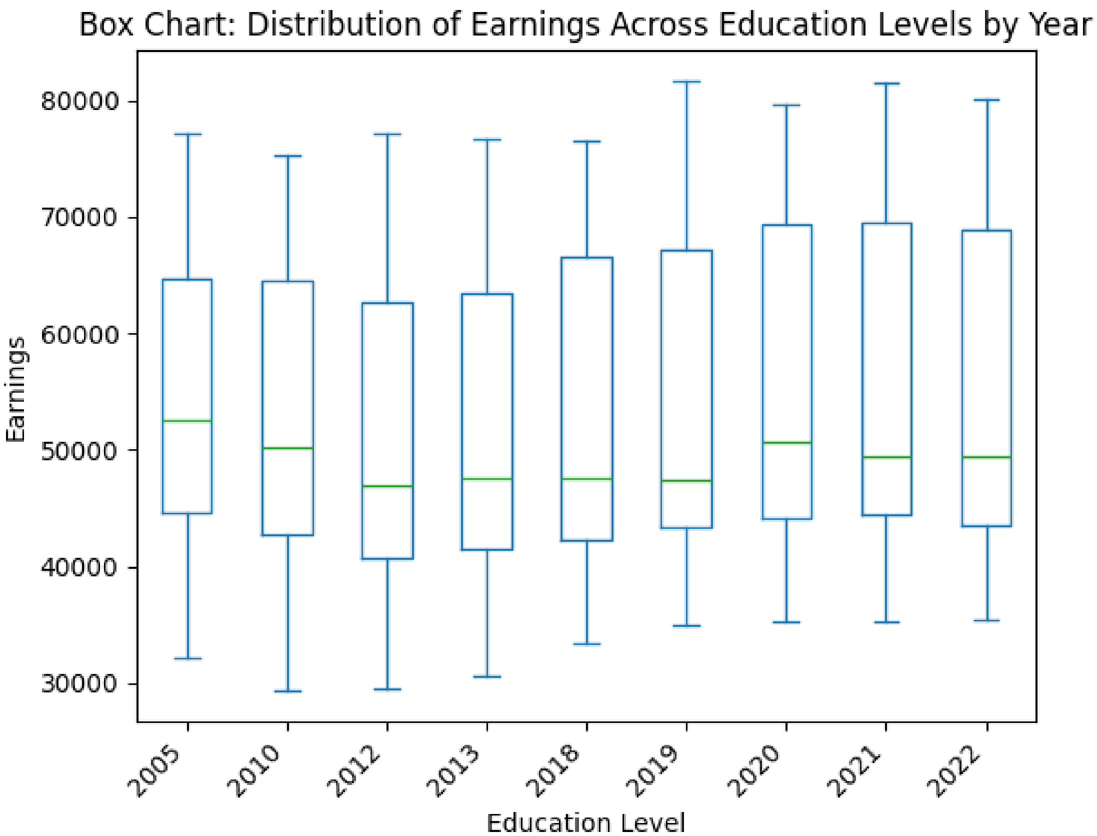


Box plot: This plot essentially gives a visual representation of the summary statistics that are generated by describe. The main question it answers is: "How does the overall spread of earnings change yearly?". Firstly, I mentioned previously that the max from 2018 to 2019 had the most significant jump, and that is visually evident by the visualization. This further demonstrates the earnings gap becoming wider over time with inflation. Moreover, the top whiskers represent the highest levels of education and vice versa. There is a clear trend of the boxes becoming wider as time passes, showing an increased spread, showing that the inequality continues to grow between education levels, further supporting the positive correlation between educational attainment and earnings found from the bar plot. Also, take note of how the median (green line) starts as almost centered from 2005, but gets increasingly further away from the center of the bar relative to its height. This also shows the increasing inequality as time passes.

[82]:
```
annual_earnings_by_education_attainment.iloc[1:8].set_index('Sex, race/
↪ethnicity, and educational attainment')[yearly_earnings_cols].
↪plot(kind='box', xlabel="Education Level", ylabel='Earnings')
```

```
plt.title('Box Chart: Distribution of Earnings Across Education Levels by Year')

plt.xticks(rotation=45, ha= 'right')
plt.show()
```



Scatter Plot: This visualization answers the same question as the bar plot, but shows overlap with the clusters. This visualization only uses a single year for the purpose of demonstration, but I have checked each of the other years, and the conclusions remain the same, with each year showing the same trend with little difference. If every individual year were included, it would be a cluster with an overall positive slope with a line of best fit. Similar to the bar plot, the Associate's level onwards shows exponential growth in earnings. Overall, This further supports the positive correlation between education level and earnings shown in the bar plot.

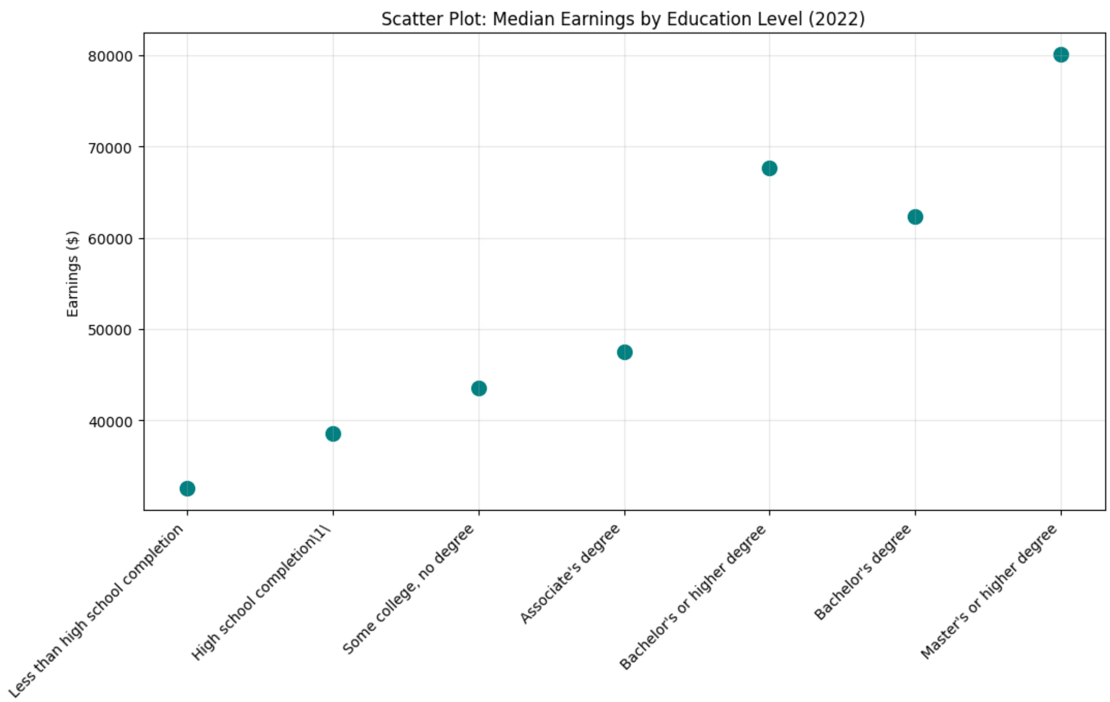
[83]:
```
import seaborn as sns
# Scatter Plot
scatter_df = annual_earnings_by_education_attainment.iloc[1:8].copy()
plt.figure(figsize=(12, 6))

# range creates a list of ints from 0 to len. This is needed to turn the␣
↪qualitative labels into numbers that matplotlib can use
```

```
plt.scatter(range(len(scatter_df)), scatter_df['2017'], s=100, color='teal')

# Place actual row labels on plot in place of placeholder ints
plt.xticks(ticks=range(len(scatter_df)), labels=scatter_df['Sex, race/
 ↪ethnicity, and educational attainment'], rotation=45, ha='right')

plt.title('Scatter Plot: Median Earnings by Education Level (2022)')
plt.ylabel('Earnings ($)')
plt.grid(True, alpha=0.3)
plt.show()
```



Error bars: This visualization answers the question: "Are the trends signified by the data completely valid, or questionable?" If you were to extend or shorten each bar to the max/min length based on the std error, the trend clearly remains consistent. For example, the highest std error whisker for 'less than high school completion' is not within the range of the lowest bar for 'high school completion', and this trend is largely consistent. With this observation, there is no doubt whatsoever that the previously derived conclusions were valid, as the extended regions of the bar maintain the same hierarchy present in the normal bar plot. Overall, the visualization reinforces the previously established positive correlation between median inc and education level from the bar plot and makes the hierarchy between levels more apparent.

[84]:
```
# Error bars
# Simple subset of original df (All rows before percents)
```

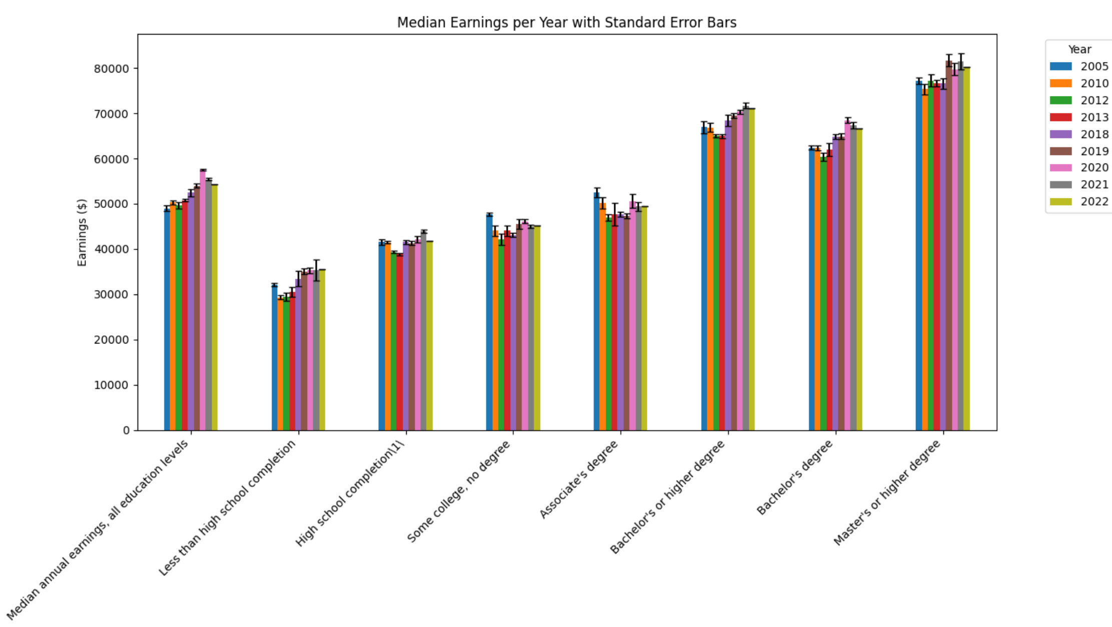
```python
subset = annual_earnings_by_education_attainment.iloc[:8].set_index('Sex, race/
↪ethnicity, and educational attainment')

# Create new df with a subset containing the yearly cols only
earnings_data = subset[yearly_earnings_cols]

# Make a copy of the error columns and equalize column names to plot data
error_values = subset[std_error_cols].copy()
error_values.columns = yearly_earnings_cols

# Pass the error_values dataframe to the yerr argument
# yerr adds whiskers, capsize adds horizontal bar to whiskers, figsize changes␣
↪width and height
earnings_data.plot(kind='bar', yerr=error_values, capsize=3, figsize=(14, 8),␣
↪xlabel="Education Level", ylabel='Earnings ($)'
)

# Make title, format ticks
plt.title('Median Earnings per Year with Standard Error Bars')
plt.xticks(rotation=45, ha='right')
plt.legend(title="Year", bbox_to_anchor=(1.05, 1), loc='upper left') #␣
↪bbox_to_anchor= and loc adjust legend position
plt.tight_layout() # Ensures no overlap of elements
plt.show()
```



Line chart: This visualization answers the question: "Is the hierarchy between earnings at ascending education levels maintained over time?". If two lines (education levels) were to ever intersect at any point, it would place previous findings into question, as there could be nuance in certain years, making the conclusion inconsistent. Thankfully, the line chart maintains the stability of the hierarchy established previously, as the lines representing the different education levels never intersect at any point, further strengthening the positive correlation between education level and earnings from the bar plot. This visualization also has the benefit of making the exponential growth from Associate's to higher education levels much more visually obvious, as there is a wide gap between the top 3 education levels and the bottom 4.

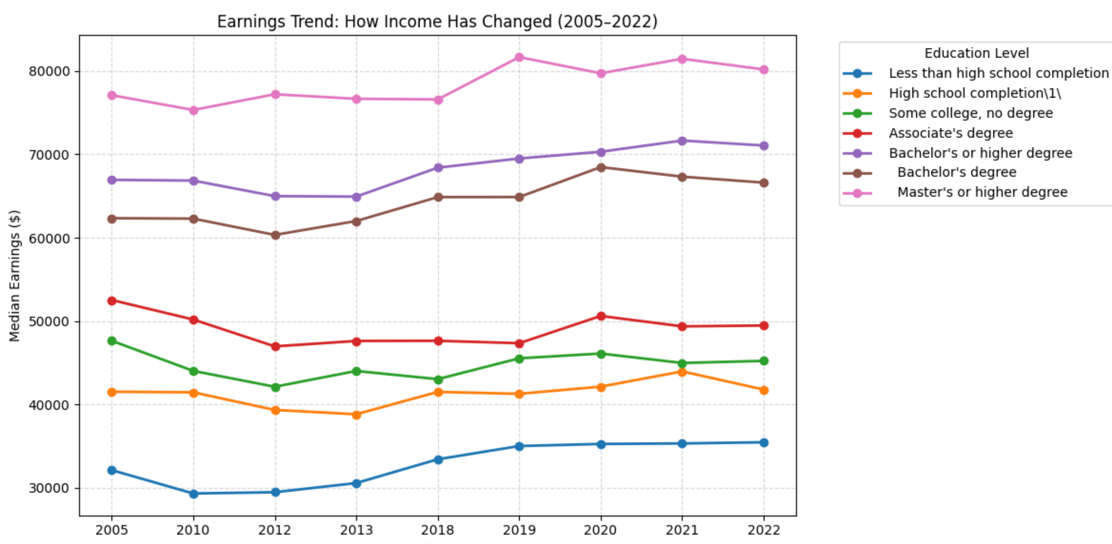
[85]:
```python
# Line chart
line_df = annual_earnings_by_education_attainment.iloc[1:8].set_index('Sex, race/ethnicity, and educational attainment')[yearly_earnings_cols]

# Use transpose to flip the table to make years the rows (x-axis) and Degrees columns (the lines)
line_df_flipped = line_df.transpose()

line_df_flipped.plot(kind='line', marker='o', figsize=(12, 6), linewidth=2)

# Formatting polish
plt.title('Earnings Trend: How Income Has Changed (2005-2022)')
plt.xlabel('Year')
plt.ylabel('Median Earnings ($)')
plt.grid(True, linestyle='--', alpha=0.5)
plt.legend(title='Education Level', bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
plt.show()
```



7. Ethical Check → "Check for bias and errors" Some ethical considerations of concern include bias and errors in the data. Luckily, these are addressed by the fact that the data is sourced from a government agency. Additionally, the standard errors for all data collected were supplied, allowing one to examine potential inaccuracies with ease and verify that derived conclusions are valid. There is also potential bias from 2022 having no standard error data, which could theoretically make the conclusions biased if there were massive margins of error. However, even in the event that happens, the range of 2005-2021 establishes such a strong trend that this is largely insignificant in the bigger picture. One must also consider nuance, as education level doesn't automatically mean higher earnings by default. Job requirements have become more stringent over time, especially in the tech industry, so even if someone has a higher education, they could still struggle to secure higher-paying roles. In addition, students by large incur thousands of dollars in debt to pursue higher education, with these costs growing exponentially depending on factors like going in versus out-of-state and the caliber of university. When these costs add up, one could question if these students have truly gained much wealth once the debt subtracted out, and the hierarchy could even break with this factored in.

8. Present Findings → "Communicating and visualizing the results" Done on the final presentation. Additional commentary: There is a clearly established positive correlation between earnings and education level, but as discussed before, there is nuance. For example, inflation must be factored in when interpreting these results, as it erodes purchasing power over time, and has a marginally larger effect on lower earnings labor than higher earnings labor. If this were theoretically measured and could show the unseen effects like a lower income worker being priced out of housing, then the disparity between levels would surely grow wider.

9. Conclusion The visualizations have painted a clear positive correlation between earnings and education attainment. This correlation was tested rigorously to expose any potential areas of nuance of questioning with potential errors. However, the hierarchy in earnings at ascending levels in education proved to be 100% consistent through the data. With this, in response to the question "How does the level of educational attainment affect median income across U.S. states?", it may be concluded that earnings generally increase with higher education.